SAVEETHA SCHOOL OF ENGINEERING
C PROGRAMMING ASSIGNMENT – Logic Analysis and Design Challenges

Field	Entry
Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Logic Analysis and Design Challenges (Q1–Q7, 50 marks)
GitHub Repository URL	https://github.com/bavadarini28 /CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

Q5 – Count the Work Done by a Loop

Deriving the loop from the principle

If n = a × b is composite, one factor satisfies a ≤ √n. So a composite n always has a divisor i with i × i ≤ n; it is enough to try i = 2, 3, … while i*i <= n, which needs no sqrt(). The loop stops early with return 0 at the first i for which n % i == 0; if the loop ends, no divisor exists and n is prime. checks++ is placed immediately before each division.

Complete the Code – TODO 1

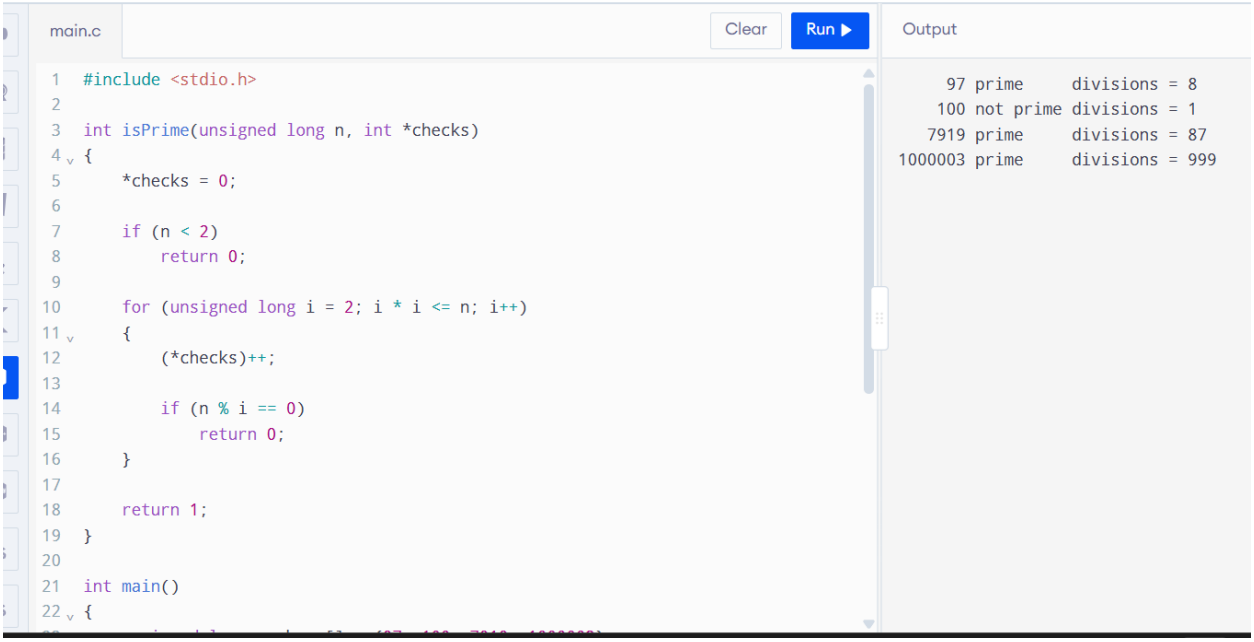
```
  for (unsigned long i = 2; i * i <= n; i++) {
     checks++;
     if (n % i == 0) return 0;
  }
  return 1;
```

Output:

```
    97 prime     divisions = 8
   100 not prime divisions = 1
  7919 prime     divisions = 87
1000003 prime     divisions = 999
```

Hand check: 97 → i = 2…9 (9×9 = 81 ≤ 97, 10×10 = 100 > 97) = 8; 1000003 → i = 2…1000 = 999.

Programiz
C Online Compiler



n	Result	Improved for	while	do–while	i < n version
97	prime	8	8	8	95
100	not prime	1	1	1	1
7919	prime	87	87	87	7,917
1,000,003	prime	999	999	999	1,000,001

```
/* while */      unsigned long i = 2; while (i <= n / i) { checks++; if (n % i == 0) return 0; i++; }
/* do-while */   if (n < 4) return n >= 2;   /* guard BEFORE the loop */
                 unsigned long i = 2; do { checks++; if (n % i == 0) return 0; i++; } while (i <= n / i);
```

Inquiry

- **do–while with n = 2 or 3:** the body runs once before the condition is tested. For n = 2, 2 % 2 == 0, so 2 is wrongly reported not prime (measured: 0). For n = 3 the answer 1 is right only by luck, after a needless division. Hence the guard n < 4 before the loop.
- **Overflow of i * i:** with a 32-bit unsigned type and n near 4,294,967,291, when i reaches 65,536, i*i = 2^32 wraps to 0 (measured), so i*i <= n stays true and the loop gives wrong answers or runs on. i <= n / i never overflows. On 64-bit unsigned long it only happens for n above about 1.8 × 10^19, but the safe form costs nothing.

Changing the condition to i < n gives 95, 1, 7,917 and 1,000,001 divisions (table above): linear in n instead of √n.

Alternative Concept and Complexity Analysis

```
char *comp = calloc(N+1, 1);
for (unsigned long i = 2; i*i <= N; i++)
   if (!comp[i]) for (unsigned long j = i*i; j <= N; j += i) comp[j] = 1;
/* query: prime iff n >= 2 && !comp[n] */
```

Sieve: O(N log log N) once (measured 2,122,051 marking operations for N = 1,000,003; 78,499 primes found), O(1) per query, O(N) space (1,000,004 bytes). Trial division: O(√n) per query (999 divisions for the worst ticket), O(1) space. For a few tickets trial division is cheaper (999 vs 2.1 million operations); for many queries

the sieve wins – e.g. testing every number up to 1,000,003 takes 67,741,504 divisions by trial division, but only about 2.1 million marks with the sieve.